Table of Contents:

Part I. GTOnline Data Types
Data Types

Part II. GTOnline Constraints
Business Logic Constraints

Part III. Task Decomposition with Abstract Code:
Log In
View Main Menu Information
View and Manage Holiday
View Audit Log
View All Manufacturers
View Category
View Actual vs Predicted GPS
View Air Conditioners on Groundhog Day
View Store Revenue by Year by State
View District Volume by Category
View Revenue by Population

Part I. GTOnline Data Types

Data Types:

PRODUC¥T

Attribute	Data Type	Nullable
PID	String	Not Null
Name	String	Not Null
Retail_price	Float	Not Null
Manufacture	String	Not Null
Categories	Collection < String >	Not Null

DISCOUNT

Attribute	Data Type	Nullable
PID	String	Not Null
Discount_date	Date	Not Null
Discount_price	Float	Not Null
Discount_sold	Integer	Not Null

MANUFACTURER

Attribute	Data Type	Nullable
Name	String	Not Null

CATEGORY

Attribute	Data Type	Nullable
Name	String	Not Null

SOLD_INFO

Attribute	Data Type	Nullable
PID	String	Not Null
Date	Date	Not Null
Quantity	Integer	Not Null

Store	String	Not Null
Sold_price	Float	Not Null

USER

Attribute	Data Type	Nullable
Employee_id	String	Not Null
Can_view_audit_log	Boolean	Not Null
Title	String	Null
Last_SSN	String	Not Null
Last_name	String	Not Null
First_name	String	Not Null
Districts	List< String >	Not Null

STORE

Attribute	Data Type	Nullable
Store_number	String	Not Null
Phone_number	String	Not Null
Store_address	String	Null
City_id	String	Not Null

CITY

Attribute	Data Type	Nullable
City_id	String	Not Null
City_name	String	Not Null
State	String	Not Null
Population	Integer	Not Null
District_number	String	Not Null

DISTRICT

Attribute	Data Type	Nullable
District_number	String	Not Null

HOLIDAY

Attribute	Data Type	Nullable
Date	Date	Not Null
Name	String	Not Null

AUDIT LOG

Attribute	Data Type	Nullable
Timestamp	Date	Not Null
Report_name	String	Not Null
User_id	String	Not Null

REPORT

Attribute	Data Type	Nullable
Report_name	String	Not Null

Part II. GTOnline Constraints

Business Logic Constraints:

PRODUCT Constraints

- Each user is uniquely identified by a PID, which consists of 12 digits and may contain leading zeros and is stored as a string.

DISCOUNT Constraints

- If a product is discounted for multiple days in a row, then a record is stored in the data warehouse for each day.
- A product can't be discounted multiple times on the same day with different discounted prices.
- Discount_price and Discount_sold must be larger than zero.

SOLD_INFO Constraints

- Sold_price and quantity must be larger than zero.

USER Constraints

- Each user is uniquely identified by an Employee_id, which can be up to 7 characters long and may contain leading zeros.
- Users are authenticated by entering the last four digits of their social security number plus a dash and their last name in the correct capitalization.
- Each user is assigned to one or more regions. Users with access to all regions can add holidays to the data warehouse.
- Audit logs of users viewing report data are stored in the database. The ability to view the audit log is controlled by the user's permissions, and there is a corresponding identification field (Can_view_audit_log) in the user table.

STORE Constraints

- The phone number should follow the general rule and format of US telephone number, that is, consist of ten digits organized into a three-digit area code, a three-digit central office code (or exchange code), and a four-digit line number.

CITY Constraints

- For the State attribute, the city should be located within one of the 50 states currently comprising the United States.
- For the Population attribute, it should be an integer greater than 0.

DISTRICT Constraints

- For access constraint, only authorized users can get access to the corresponding district information.

Part III. Task Decomposition with Abstract Code:

Login

Task Decomp

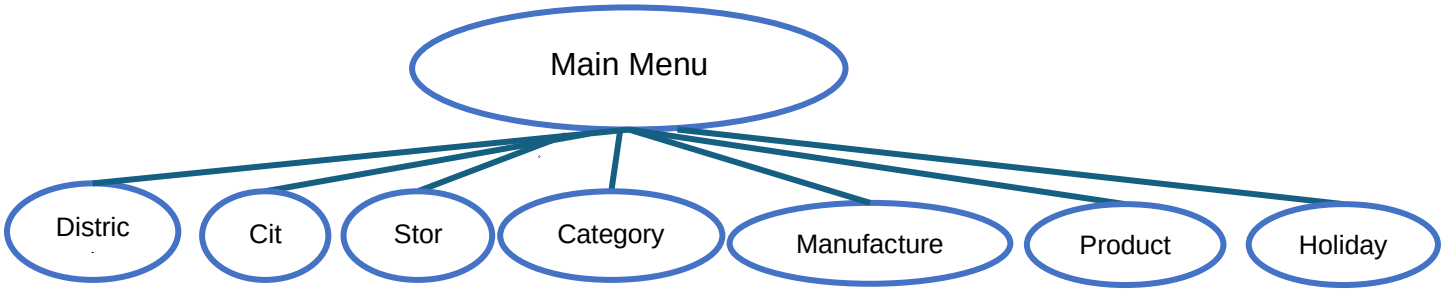
Lock Types: read-only on USER
Number of Locks: **1** read lock (USER)
Enabling Conditions: Can be accessed by all users.
Frequency: Very high. All users, no matter what tasks are related, need to log in first to make further actions.
Consistency (ACID): Not critical. Order is not critical. Users' credentials are maintained by DBA and cannot be changed by users at any time.
Subtasks: Neither mother tasks nor subtasks are required.

Abstract Code

- User enters Employee ID ('$EmployeeID'), password ('$Password') input fields.
- If data validation is successful for both Employee ID and password input fields (Starting with 4 digits followed by a dash and capitalized letters), then:
    - When ***Login*** button is clicked, run the **Login** task:
        - If User record is found but USER.Credentials.Last_SSN + '_' + USER.Credentials.Last_NAME != $Password:
            - Go back to **Login** form, with error message.
    - Else:
        - Store login information as session variable $UserID.
        - Go to the **Main Menu** list.
- Else Employee ID and password input fields are invalid, display **Login** form, with error message.

View Main Menu Information

Task Decomp



Lock Types: Read-only on DISTRICT, CITY, STORE, CATEGORY, MANUFACTURER, PRODUCT, HOLIDAY.

Number of Locks: 7 read-only locks.

Enabling Conditions: Can be accessed by all users.

Frequency: High.

Consistency (ACID): Critical. Consistency is important as it should be able to calculate the statistics of other pages mentioned in subtasks.

Subtasks: Access to other pages and calculate the statistics of other pages (as shown here, the main menu needs to access DISTRICT, CITY, STORE, CATEGORY, MANUFACTURER, PRODUCT and HOLIDAY).

Abstract Code

The Main menu will serve as an interface for other pages.

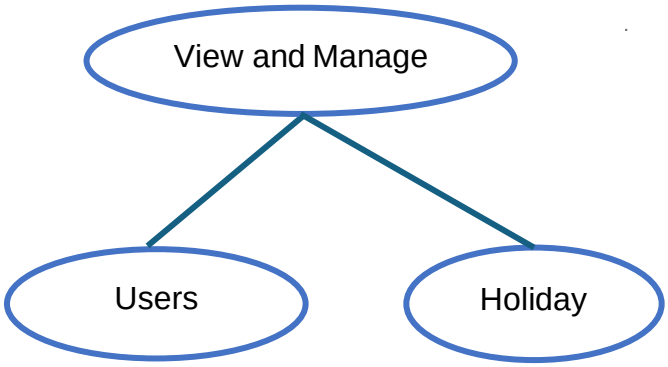
- Match the users stored according to the employee ID and password provided in **Log in Form**.
  - Valid user: give him/her access to log into the system
  - Invalid user: ask him/her to recheck the employee ID and password
- Display welcome information for the user who logs into the system by getting the USER.First and USER.Last name of users, and calculate the statistics of other pages

(the count of stores, cities, districts, manufacturers, products, categories, and holiday) in the database at the time of log in.

- Display links for all the pages within the system, and if the user clicks certain link, then jump to the corresponding page.

View and Manage Holiday

Task Decomp



Lock Types: view: Read-only on HOLIDAY; Manage: Read-only for USER and Write for HOLIDAY.

Number of Locks: View: 1 read lock; Manage: 1 read lock and 1 write lock.

Enabling Conditions: "View" can be accessed by all users, while "Manage" can only be accessed for Corporate Users.

Frequency: Medium.

Consistency (ACID): Non-critical. Holiday stored in chronological order.

Subtasks: All tasks must be done but can be done in parallel. All users can view the HOLIDAY form, but only corporate users have the write permission. So for the 'Manage' task, the subtype of users should be checked first.

Abstract Code

(1) Default users only have 'View' permission:

- User clicked on ***View Holiday*** button from **Main Menu**:
- There will be two subtasks: 'View Holiday' and 'Manage Holiday'. For default users, they cannot get response if they select subtask 'Manage Holiday'

- Run the **View Holiday** task: display the Date and Name of the current Holiday stored in the Holiday Management Form in chronological order.

(2) For corporate users who have 'Manage Holiday' permissions:

Besides the 'View' permission mentioned above, they can manipulate the 'Manage Holiday' subtask. There will be 3 manipulations for this subtask.

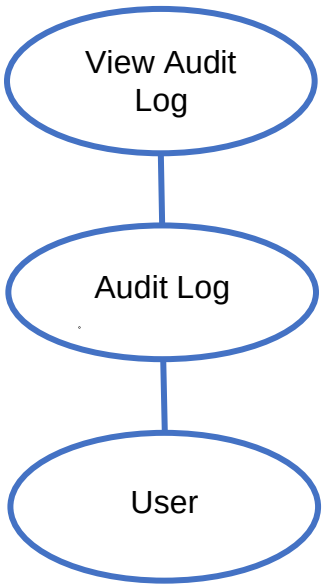
- For 'Add', the corporate user can add a new row into Holiday Management Form, and the user needs to provide the Date and Name of the added holiday.
- For 'Delete', the corporate user can select and delete one or multiple rows in the Holiday Management Form.
- For 'Modify', the corporate user can modify the Date and Name of the existing data in the Holiday Management Form.

After these manipulations, the relevant information, such as the employee ID, manipulation time, and the changes the corporate user made, will be recorded. Then the updated Holiday Management Form will be sorted again in chronological order.

- When ready, user selects a next action from choices in **Main Menu**.

View Audit Log

Task Decomp



Lock Types: Read-only on AUDIT_LOG, USER

Number of Locks: 2 read locks.

Enabling Conditions: Triggered by clicking view audit log on main menu.

Frequency: low. User has higher frequency than Audit Log as they are in 1-Many relationships. Only be accessed by users permission.

Consistency (ACID): Not critical. Audit Log can not be revised.

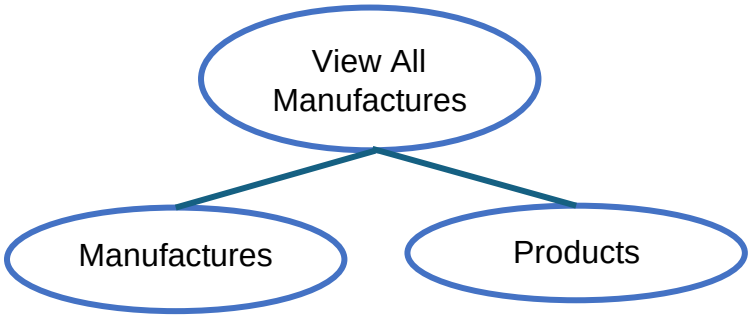
Subtasks: All tasks must be done, and Audit Log must be retrieved first before getting to the User who created it. Order is necessary.

Abstract Code

- User clicked on ***View Audit Log*** button from **Main Menu**:
- Run the **View Audit Log** task: query for the most recent 100 audit log records.
    - Find the most recent 100 audit logs using the AUDIT_LOG. Date; Display the AUDIT_LOG.Timestamp and AUDIT_LOG.Report_name.
    - Find the USER where AUDIT_LOG.EmployeeID == USER. EmployeeID; Display User.Credential.Last_name and User. Credential. First_name separated by a comma and space.
    - If the USER is a FULL_ACCESS_USER where USER.Disctrict.length() == DISCTRICT.length(), which means the USER has been assigned to all districts, mark the row with a yellow background.
- When ready, user selects a next action from choices in **Main Menu**.

View All Manufacturers

Task Decomp



Lock Types: Read-only on MANUFACTURER, PRODUCT

Number of Locks: 2 read locks.

Enabling Conditions: Triggered by clicking view all manufactures on main menu.

Frequency: Medium. Can be accessed by all users.

Consistency (ACID): Critical. If the product's retail price is changed, it will affect the query result.

Subtasks: All tasks must be done but can be done in parallel. Mother task is required to coordinate subtasks. After getting all Products made by a specific Manufacturer, sort all Manufactures by the average price in a descending order and keep only the top100. Order is necessary.

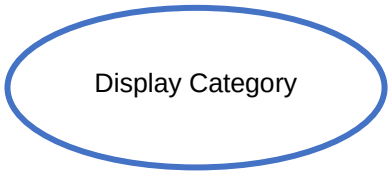
Abstract Code

- User clicked on ***View Manufactures*** button from **Main Menu**:
- Run the **View All Manufactures** task: query for all manufactures and their products.
    - Find all products. Group products according to PRODUCT.Manufacture.
    - Find all manufactures. For each Manufacture:
        - Display the $Total_products as the total number of products it manufactures where PRODUCT.Manufacture == MANUFACTURE.Name.
        - Calculate $Average_retail_price of all these products based on PRODUCT.Retail_price.
    - Sort Manufactures by the $Average_retail_price in a descending order; Keep only top 100 manufactures.
        - For each Manufacture:
            - Display the MANUFACTURE. Name.
            - Display the $Average_retail_price
            - Find the $Min_retail_price, and $Max_retail_price based on PRODUCT.Retail_price; Display the $Min_retail_price, and $Max_retail_price.
- When user clicks on each ***Manufacture Row***:
    - Display MANUFACTURE. Name, $Total_products, $Average_retail_price, $Min_retail_price, and $Max_retail_price.
    - Find all PRODCUT that are manufactured by this manufacture;
    - For each Product:
        - Find PRODUCT.Retail_price of each product, and order all products by price descending.

        - Display PRPDUCT.PID, PRPDUCT.Name,
        - Find all categories this product belongs to with PRODUCT.Categories; Display a list of categories with a comma and space delimiting each category.
- When ready, user selects a next action from choices in **Main Menu**.

View Category

Task Decomp



Lock Types: Read-only on CATEGORY, MANUFACTURER, PRODUCT
Number of Locks: 3 read locks

Enabling Conditions: Triggered by clicking Display Category on main menu.
Frequency: Generally low, can be accessed by all users. On-demand when the report is requested.
Consistency (ACID): Not critical. Order is not critical.
Subtasks: Mother Task is not needed. No decomposition needed.

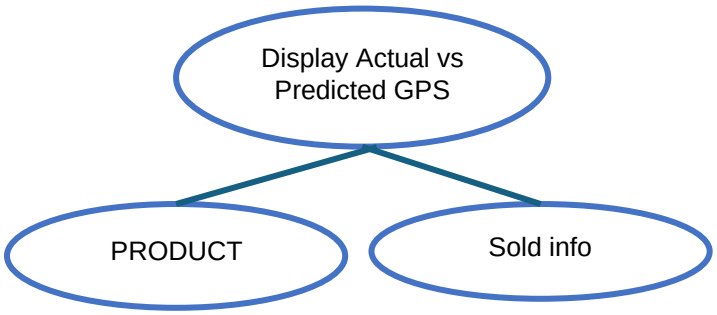
Abstract Code

- User clicked on **Category Report** button from **Main Menu**:
- Run the **Display Category** task: Query a sorted (ascending) list of all CATEGORY.Name in the database.
    - For each CATEGORY.Name:
        - For each CATEGORY.Name:
        - Retrieve a list of PRODUCT (PRODUCT.PID,) categorized by CATEGORY.Name and their respective counts. Display the CATEGORY.Name and counts in the **Category Report**.
        - Query the database to find the count of different Manufacturer associated with these PRODUCT using the PRODUCT.PID and MANUFACTURER.Name. Display this count in the **Category Report**.
        - Query the database to find the average PRODUCT.Retail_price for all of these PRODUCT. Display on the **Category Report**.

- When ready, user selects next action from choices in **Main Menu**.

View Actual vs Predicted GPS

Task Decomp



Lock Types: read-only on PRODUCT, SOLD_INFO, CATEGORY
Number of Locks: 3 read locks
Enabling Conditions: Triggered by clicking Display Actual vs Predicted GPS on main menu.
Frequency: Generally low, can be accessed by all users. On-demand when the report is requested.
Consistency (ACID): Critical. Consistency is important as it should be able to calculate the statistics mentioned in subtasks. Order is not critical.
Subtasks: All tasks must be done but can be done in parallel. Mother task is required to coordinate subtasks. Order is not necessary.

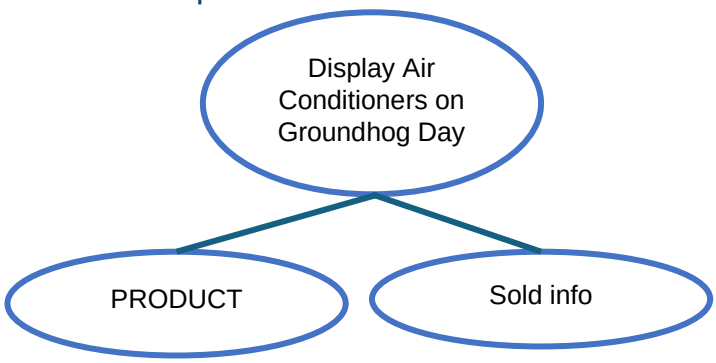
Abstract Code

- User clicked on **GPS Report** button from **Main Menu**:
- Run the **View Actual vs Predicted GPS** task: Query the database for a list of all CATEGORY.Name = ”GPS”. For each PRODUCT.PID:
  - Set:
    - $TOTAL _RETAIL_SALES = SUM(SOLD_INFO.Quantity) - SUM (PRODUCT.Discount_sold)
    - $ACTUAL_REVENUE = $TOTAL _RETAIL_SALES * PRODUCT.Retail_price + SUM(PRODUCT.Discount_sold * PRODUCT.Discount_price)
    - $EXPECTED_REVENUE = $TOTAL _RETAIL_SALES * 0.75
    - $ACTUAL_EXPECTED_DIFFERENCE = $ACTUAL_REVENUE - $EXPECTED_REVENUE
  - Display in **Category Report**:
    - PRODUCT.PID
    - PRODUCT.Name

    - PRODUCT.Retail_price
    - SOLD_INFO.Quantity
    - PRODUCT.Discount_sold
    - $TOTAL _RETAIL_SALES
    - $ACTUAL_REVENUE
    - $EXPECTED_REVENUE
    - If $ACTUAL_EXPECTED_DIFFERENCE > 200, display $ACTUAL_EXPECTED_DIFFERENCE and sorted in descending order
- When ready, user selects next action from choices in **Main Menu**.

View Air Conditioners on Groundhog Day

Task Decomp



Lock Types: read-only on SOLD_INFO, PRODUCT, CATEGORY
Number of Locks: 3 read locks
Enabling Conditions: Triggered by clicking Display Air Conditioners on Groundhog Day on main menu.
Frequency: Generally medium. On-demand when the report is requested.
Consistency (ACID): Critical. Consistency is important as it should be able to calculate the statistics mentioned in subtasks.
Subtasks: All tasks must be done but can be done in parallel. Mother task is required to coordinate subtasks. Order is not necessary.

Abstract Code

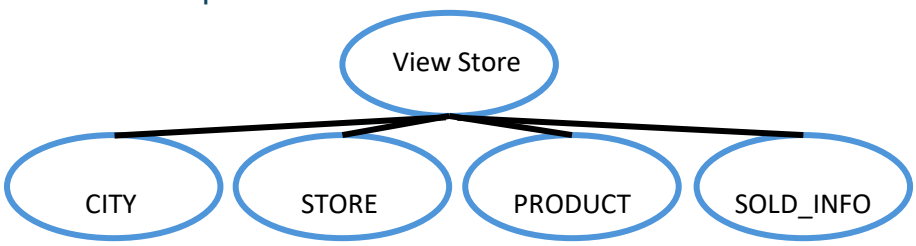
- User clicked on **Air Conditioners on Groundhog Day Report** button from **Main Menu**:
- Run the **View Air Conditioners on Groundhog Day** task: Query the database for a list of all CATEGORY.Name = ”Air Conditioner”, sorted by SOLD_INFO.Date.Year in ascending order.
    - For each SOLD_INFO.Date.Year:

- Retrieve the total quantity ($TOTAL_QUANTITY = SUM(SOLD_INFO.Quantity)) of the PRODUCT sold in the year categorized by CATEGORY.Name = "Air Conditioner"
- Set $AVE_PER_DAY = $TOTAL_QUANTITY / 365.0
- Get the number of air conditioner products sold on February 2 of that year ($GHD_QUANTITY = SUM(SOLD_INFO.Quantity)) of products (CATEGORY.Name = "Air Conditioner" during that year, and the day is February 2)
- Display SOLD_INFO.Date.Year, $TOTAL_QUANTITY, $AVE_PER_DAY, $GHD_QUANTITY in **Category Report**.

- When ready, user selects a next action from choices in **Main Menu**.

View Store Revenue by Year by State

Task Decomp



Lock Types: read-only on CITY ,STORE, SOLD_INFO, PRODUCT
Number of Locks: 4 read locks
Enabling Conditions: Triggered by users with full access to all the districts clicking the ***Enter*** button or *hyperlink* of **Store Revenue by Year by State Report** on **Main Menu**.
Frequency: Low. Only users with access to all the districts can view this corporate report.
Consistency (ACID): Critical. Order is critical. Need to check CITY first, if there is available STORE information then check SOLD_INFO, within SOLD_INFO there is a derived attribute so might need to compute based upon STORE data.
Subtasks: Mother task is required. Subtasks need to be done in order. CITY->STORE->SOLD_INFO->PRODUCT.

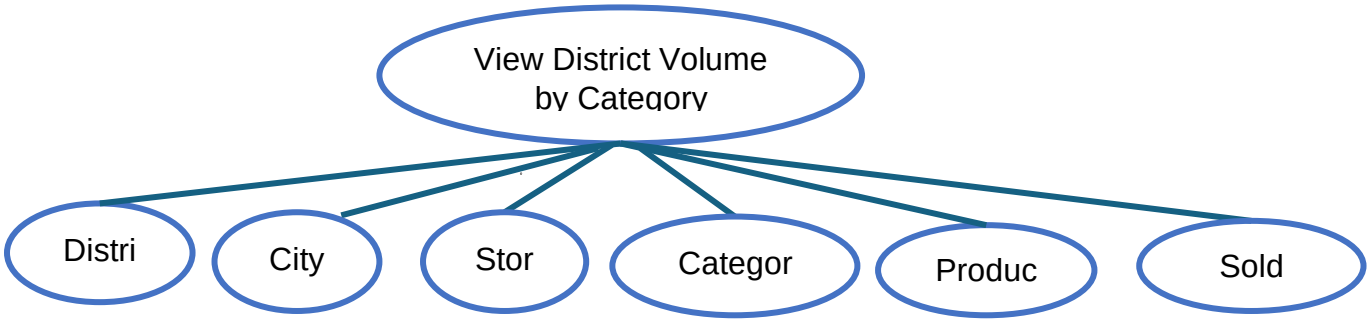
Abstract Code

- When the button or *hyperlink* of **Store Revenue by Year by State Report** on **Main Menu** is clicked, run the **View Store Revenue by Year by State** task.
- Wait for the user to select a state from the *drop-down box* and click ***Enter*** button. Once the ***Enter*** button is clicked:

- Retrieve all STORE LOCATE IN CITY where CITY.state = '$state'.
- Initializes an empty list of results.
- For each STORE:
  - Retrieve STORE.Store_number, STORE.Store_address, CITY.City_name, and "Annual Revenue".
- Calculate the "Annual Revenue" of each year of each STORE:
  - Initialize "Annual Revenue" to 0.
  - For each PRODUCT SOLD AT each STORE in each SOLD_INFO.Date.year:
    - If SOLD_INFO.DATE does not match PRODUCT.Discount_date:
      - Revenue += PRODUCT.retail_price * SOLD_INFO.Quantity
    - If SOLD_INFO.DATE does match PRODUCT.Discount_date:
      - Revenue += PRODUCT.Discount_price * SOLD_INFO.Quantity
  - Add "Annual Revenue" and SOLD_INFO.Date.year AS "Year" to the results list.
- Sort the results list by Year in ascending order.
- Sort the results list by Revenue in descending order (2nd level).
- Display the sorted results to the user.
- When ready, user selects next action from choices in **Main Menu**.

View District Volume by Category

Task Decomp



Lock Types: Read-only on DISTRICT, CITY, STORE. CATEGORY, PRODUCT, SOLD_INFO.

Number of Locks: 6 read locks.

Enabling Conditions: Triggered by users with full access to all the districts (i.e., corporate users) clicking the **District Volume for Category Report,** and providing the specific period that the users want to conduct the analysis

Frequency: Low. Only corporate users have this permission, and the report will usually be generated monthly.

Consistency (ACID): Critical as it involves the calculation of many forms. And the order is critical too, but there may be several strategies to solve this problem.

Subtasks: As mentioned before, there are several strategies to solve this problem. Here I would suggest a bottom-to-top strategy, that is, given a certain period, we may first extract all the product sales in each store, and then further calculate the sales volume of each category in each store. Subsequently, we can calculate the sales volume of each category in each district. Finally, for each district, we rank the categories by their sales volume in a descending order.

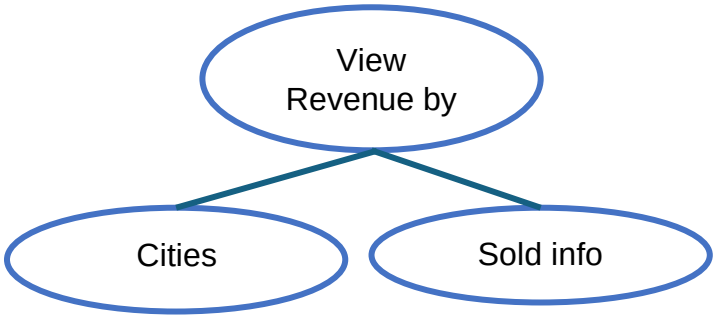
Abstract Code

According to the description of subtasks, after the corporate user has clicked the **District Volume for Category Report,** the platform should prompt the user to specify the time range that he or she wants to conduct the analysis. Then the report generation command executes.

- Extract all the sales that occurred during the given period by referring to SALE_INFO.Date attribute.
- With the extracted sales information, group them by stores, and then calculate the total quantity of each product sold at each store.
- Further calculate the total quantity of each category at each store by assigning the products into their corresponding categories.
- Assign the stores to their corresponding city and district, and calculate the total volume of each category and each product in the district.
- Group the data by category and rank the district by sales volume in descending order. Also, within each district, rank products of the same category by their sales volume in descending order.
- Generate a report containing information for each category, including details of the district that sold items in each category, as well as the specific products sold, and return it to the requesting user.
- When ready, user selects next action from choices in **Main Menu**.

View Revenue by Population

Task Decomp



Lock Types: Read-only on CITY, SOLD_INFO

Number of Locks: Several different schema constructs are needed.

Enabling Conditions: Triggered by clicking View Revenue by Population on main menu.

Frequency: low. Only available to users with access to all districts.

Consistency (ACID): Not critical, order is not critical.

Subtasks: All tasks must be done but can be done in parallel. Mother task is required to coordinate subtasks. Order is not necessary.

Abstract Code

- User clicked on ***View Revenue by Population*** button from **Main Menu**:
- Run the **View Revenue by Population** task: query for all cities, their population and revenue.
    - Find all cities, and CITY.Polulation.
    - Categorize these cities based on the population size as:
        - Small: population <3,700,000
        - Medium: 3,700,000 <= population <6,700,000
        - Large: 6,700,000 <= population <9,000,000
        - Extra Large: population >=9,000,000
    - For each city:
        - Find all stores located in this city.
        - For each store:
            - Find all sold_info where SOLD_INFO.Store == STORE.Store_number.
            - Group sold_info by year of SOLD_INFO.Date.

        - For each sold_info group:
            - Add PRODUCT.Selling_price to get annual revenue of this store.
        - Add each store's annual revenue to get the city's annual revenue.
    - For each population group:
        - Display population group name (Small/ Medium/ Large/ Extra Large).
        - Sort annual revenue of each city by year from oldest to newest and by population from smallest to largest; Display the CITY.Name, year and revenue.

IDF & EER DRAW.IO LINK:

https://drive.google.com/file/d/1Rrqe96MPQJaZRJLlegxw-mkx6KEA6f18/view?usp=sharing